Sparse-Aware RISC-V Vector Architecture for High-Performance Computing

Mid-Semester Evaluation Report — Project 1, Group 59

Sakshat Sachdeva (2023474) Ansh Verma (2023101)

October 14, 2026

Abstract

[TODO: 4–6 sentences: what was built, what was measured, the one-line finding, what comes next.]

1 Problem and Motivation

[TODO: Why vector hardware loses efficiency on sparse data; why SpMV is the chosen workload; what the project will build by the end-evaluation.]

2 Background

2.1 The RISC-V Vector Extension

[TODO: VLEN, SEW, LMUL, VLMAX; vsetvl and strip-mining; tail policy; unit-stride vs indexed loads; reductions. Cite [1].]

2.2 Sparse Formats and SpMV

[TODO: CSR (rowptr, colidx, vals); where the indirect access is; memory footprint of CSR vs dense; mention COO, ELL, SELL-C-sigma briefly. Cite [2, 3].]

2.3 Existing Sparse-Aware Architectures

[TODO: Short survey, 1–2 sentences each, with citations you have actually read (see docs/READING_LIST.md).]

3 Experimental Setup

Table 1: Simulation environment.

Compiler	riscv64-unknown-elf-gcc 16.1.0, `-O2 -march=rv64gcv_zicntr -mabi=lp64d`
Simulator	Spike 1.1.1-dev (commit 609dbe0b) with proxy kernel pk (commit 9c61d298)
ISA string	`rv64gcv_zicntr_zvl{128,256,512}b`
Vector config	SEW = 32 (float), LMUL = 1, VLMAX = 4 / 8 / 16
Matrix size	$N = 128$, seed 1 (also validated at $N = 37, 100$ and seeds 2, 3)
Sparsity levels	0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.8, 0.9, 0.95, 0.99 (target; actual value reported)
Patterns	uniform random, banded, 8×8 block
Metric	retired instructions (`rdinstret`) of the second call of each kernel
Correctness	relative error $< 10^{-3}$ against a double-precision Python reference

[TODO: One paragraph on the toolchain build (scripts/setup_tools.sh), one on how a run works (make spike), one on what Spike can and cannot measure.]

3.1 Measurement Methodology

[TODO: Explain the warm-up call and why it is needed; explain how vec_iters and avg_vl are obtained.]

Table 2: Instructions retired by the first (cold) and second (warm) call of each kernel. $N = 128$, VLEN = 512, uniform, sparsity 0.

kernel	first call	second call	ratio
scalar_csr	324036	149645	2.17
dense_rvv	95383	13458	7.09
csr_rvv	13712	13713	1.00

4 Baseline Implementations

4.1 Dense RVV GEMV

Listing 1: Dense GEMV with RVV intrinsics (src/kernels.c).

```
/* Baseline 1: dense GEMV. Zeros are multiplied anyway. */
void gemv_dense_rvv(int n, const float *A, const float *x, float *y) {
  size_t vlmax = __riscv_vsetvlmax_e32m1();
  for (int i = 0; i < n; i++) {
    const float *row = A + (size_t)i * n;
    vfloat32m1_t acc = __riscv_vfmv_v_f_f32m1(0.0f, vlmax);
    int j = 0;
    while (j < n) {
      size_t vl = __riscv_vsetvl_e32m1(n - j);
      vfloat32m1_t a = __riscv_vle32_v_f32m1(row + j, vl);
      vfloat32m1_t xv = __riscv_vle32_v_f32m1(x + j, vl);
      acc = __riscv_vfmacc_vv_f32m1_tu(acc, a, xv, vl);
      j += vl;
    }
    vfloat32m1_t z = __riscv_vfmv_s_f_f32m1(0.0f, 1);
    z = __riscv_vfredusum_vs_f32m1_f32m1(acc, z, vlmax);
    y[i] = __riscv_vfmv_f_s_f32m1_f32(z);
  }
}
```

[TODO: Walk through the loop: strip-mining with vsetvl, two unit-stride loads, vfmacc with tail-undisturbed, one reduction per row. Why _tu is required.]

4.2 CSR RVV SpMV

Listing 2: CSR SpMV with RVV intrinsics (src/kernels.c).

```
/* Baseline 2: CSR SpMV, indexed gather on x (vluxei32), one reduction per row. */
void spmv_csr_rvv(int n, const int *rowptr, const unsigned *col, const float *val,
                  const float *x, float *y) {
  size_t vlmax = __riscv_vsetvlmax_e32m1();
  for (int i = 0; i < n; i++) {
    int k = rowptr[i], end = rowptr[i + 1];
    vfloat32m1_t acc = __riscv_vfmv_v_f_f32m1(0.0f, vlmax);
    while (k < end) {
      size_t vl = __riscv_vsetvl_e32m1(end - k);
      vuint32m1_t idx = __riscv_vle32_v_u32m1(col + k, vl);
```

```
      idx = __riscv_vsll_vx_u32m1(idx, 2, vl); /* element -> byte offset */
      vfloat32m1_t xv = __riscv_vluxei32_v_f32m1(x, idx, vl);
      vfloat32m1_t vv = __riscv_vle32_v_f32m1(val + k, vl);
      acc = __riscv_vfmacc_vv_f32m1_tu(acc, vv, xv, vl);
      k += vl;
    }
    vfloat32m1_t z = __riscv_vfmv_s_f_f32m1(0.0f, 1);
    z = __riscv_vfredusum_vs_f32m1_f32m1(acc, z, vlmax);
    y[i] = __riscv_vfmv_f_s_f32m1_f32(z);
  }
}
```

[TODO: Walk through: load column indices, shift to byte offsets, indexed gather of x, load values, vfmacc, reduction. Why the vector never crosses a row boundary and what that costs.]

4.3 Scalar Reference and Correctness Check

[TODO: Scalar CSR loop; the Python-generated reference; tolerance; why the vector result differs from scalar at the 10^{-5} level (unordered reduction).]

5 Results

5.1 Instructions versus Sparsity

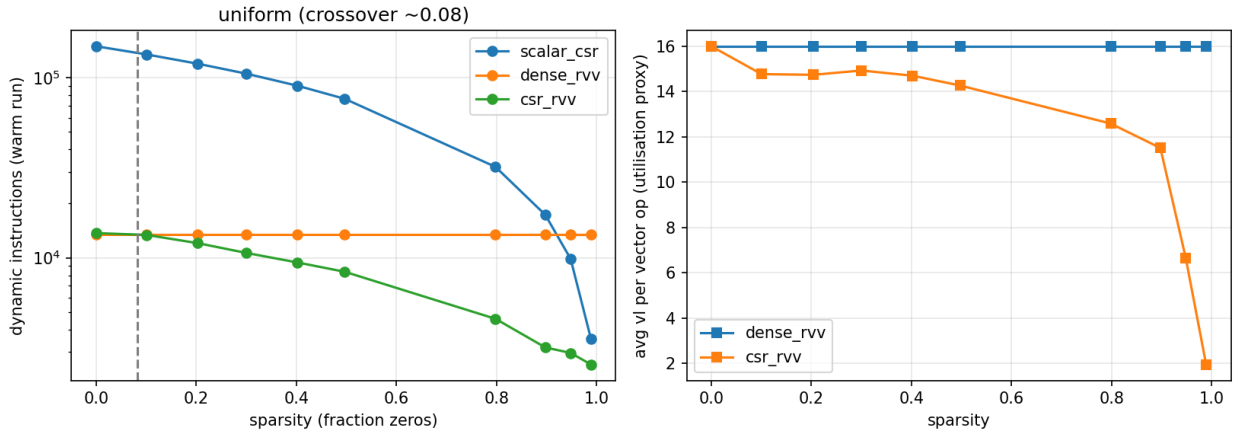


Figure 1: Uniform pattern, VLEN = 512. Left: retired instructions per kernel (log scale), dashed line marks the observed dense-to-CSR crossover. Right: average vector length per vector operation.

Table 3: Uniform pattern, VLEN = 512, $N = 128$. Instruction counts of the warm call; avg vl is for CSR RVV.

sparsity	nnz	scalar CSR	dense RVV	CSR RVV	avg vl	dense/CSR	scalar/CSR
0.000	16384	149645	13458	13713	16.00	0.98	10.91
0.100	14741	134858	13458	13401	14.77	1.00	10.06
0.203	13056	119693	13458	12057	14.74	1.12	9.93
0.300	11470	105419	13458	10641	14.93	1.26	9.91
0.402	9792	90317	13458	9417	14.70	1.43	9.59
0.497	8246	76403	13458	8361	14.27	1.61	9.14
0.798	3311	31988	13458	4581	12.59	2.94	6.98
0.897	1681	17318	13458	3177	11.51	4.24	5.45
0.948	852	9857	13458	2961	6.66	4.55	3.33
0.989	183	3530	13458	2554	1.95	5.27	1.38

[TODO: Describe what the table shows. Do not interpret yet.]

5.2 Effect of Sparsity Pattern

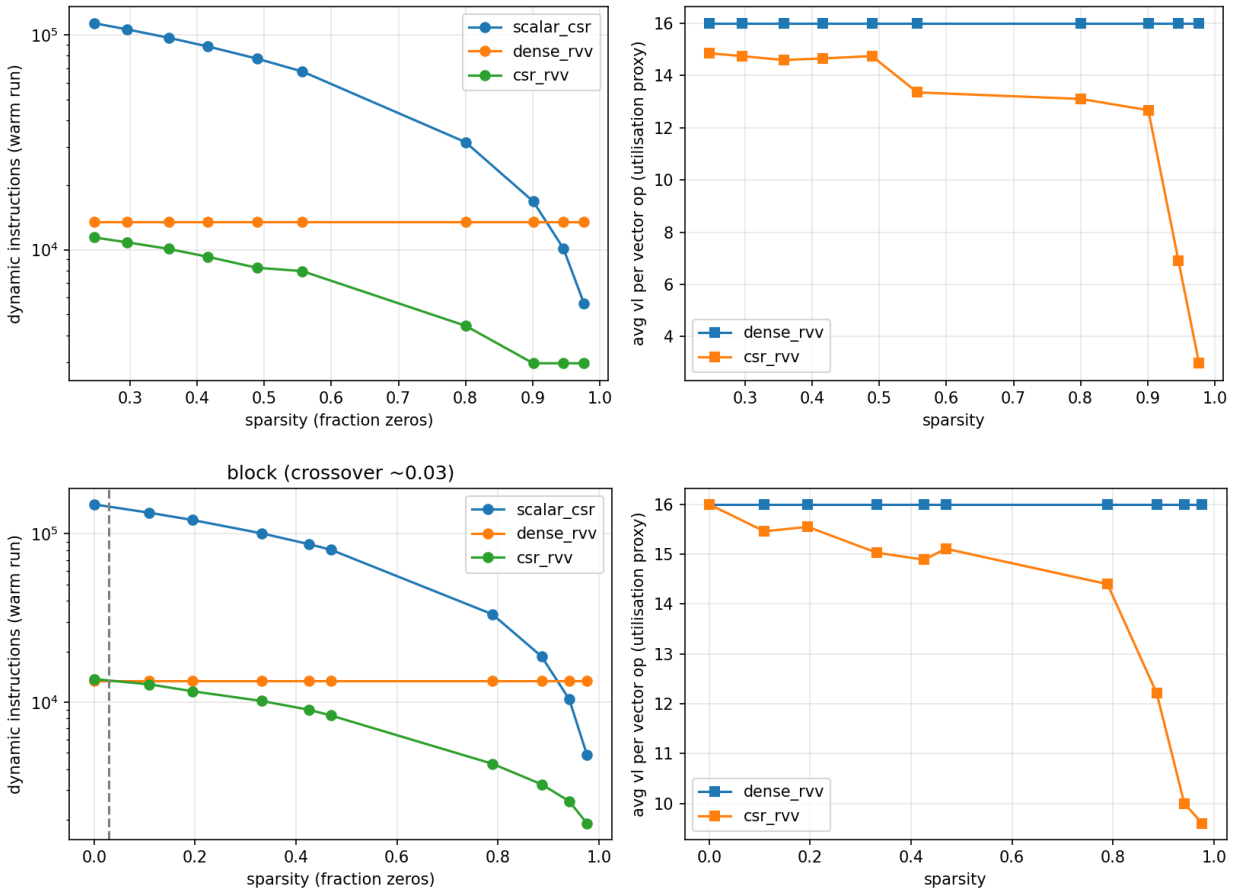


Figure 2: Banded (top) and 8×8 block (bottom) patterns, VLEN = 512.

Table 4: Block pattern, VLEN = 512.

sparsity	nnz	scalar CSR	dense RVV	CSR RVV	avg vl	dense/CSR	scalar/CSR
0.000	16384	149645	13458	13713	16.00	0.98	10.91
0.109	14592	133517	13458	12753	15.46	1.06	10.47
0.195	13184	120845	13458	11601	15.55	1.16	10.42
0.332	10944	100685	13458	10161	15.03	1.32	9.91
0.426	9408	86861	13458	9009	14.89	1.49	9.64
0.469	8704	80525	13458	8337	15.11	1.61	9.66
0.789	3456	33293	13458	4305	14.40	3.13	7.73
0.887	1856	18677	13458	3249	12.21	4.14	5.75
0.941	960	10397	13458	2577	10.00	5.22	4.03
0.977	384	4853	13458	1905	9.60	7.06	2.55

Table 5: Banded pattern, VLEN = 512. Note the banded generator cannot reach sparsity below 0.25 at $N = 128$ (band half-width is capped at $N/2$).

sparsity	nnz	scalar CSR	dense RVV	CSR RVV	avg vl	dense/CSR	scalar/CSR
0.246	12352	113357	13458	11409	14.85	1.18	9.94
0.295	11554	106175	13458	10833	14.74	1.24	9.80
0.357	10532	96977	13458	10089	14.59	1.33	9.61
0.415	9578	88391	13458	9273	14.65	1.45	9.53
0.489	8374	77555	13458	8241	14.74	1.63	9.41
0.557	7264	67565	13458	7953	13.35	1.69	8.50
0.800	3274	31655	13458	4425	13.10	3.04	7.15
0.901	1622	16787	13458	2961	12.67	4.55	5.67
0.946	884	10145	13458	2961	6.91	4.55	3.43
0.977	382	5627	13458	2962	2.98	4.54	1.90

5.3 Effect of Vector Length

Table 6: Uniform pattern: dense and CSR RVV instruction counts at VLEN = 128, 256, 512.

sparsity	dense V128	CSR V128	dense V256	CSR V256	dense V512	CSR V512
0.000	50322	50577	25746	26001	13458	13713
0.100	50322	46245	25746	24213	13458	13401
0.203	50322	41145	25746	21669	13458	12057
0.300	50322	36405	25746	19305	13458	10641
0.402	50322	31413	25746	16797	13458	9417
0.497	50322	26769	25746	14517	13458	8361
0.798	50322	11949	25746	7089	13458	4581
0.897	50322	7005	25746	4569	13458	3177
0.948	50322	4581	25746	3285	13458	2961
0.989	50322	2578	25746	2554	13458	2554

6 Analysis

[TODO: Why dense is flat. Why CSR cost falls then floors. Why avg vl collapses. Why block keeps avg vl high. Why wider vectors stop helping. Where the crossover is and why the Spike crossover is a lower bound.]

7 Limitations and Failure Cases

[TODO: Spike is not a timing model; gather cost hidden; no cache model; small N; synthetic matrices only; first measurements were wrong and why (page faults, in-loop counters), see docs/MEASUREMENT_NOTES.md.]

8 Plan for the End-Evaluation

[TODO: Chosen mechanism (software format such as SELL-C-sigma, or a modelled hardware mechanism in Spike), timing model choice (Ara/Verilator vs gem5), second workload, metrics required by the spec, schedule and roles.]

Artifact

Source, scripts, results and documentation are in the submitted archive `59_1.zip`; see `README.md`. Representative result: `make spike N=64 SPARS="0.5 0.9" PATTERNS=uniform OUT=/tmp/demo`.

References

[1] RISC-V International, "RISC-V "V" Vector Extension, Version 1.0." `https://github.com/riscv/riscv-v-spec`, 2021.

[2] N. Bell and M. Garland, "Implementing sparse matrix-vector multiplication on throughput-oriented processors," in *Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis (SC '09)*, 2009.

[3] M. Kreutzer, G. Hager, G. Wellein, H. Fehske, and A. R. Bishop, "A unified sparse matrix data format for efficient general sparse matrix-vector multiplication on modern processors with wide SIMD units," *SIAM Journal on Scientific Computing*, vol. 36, no. 5, pp. C401–C423, 2014.